

Periodic Graph Display Specification

Renderer-independent canonical-cell, local-unwrapped, and expanded graph materialization

mdstats

Contents

1 Purpose and status	2
2 Motive	2
3 Normative ownership	3
4 AI context summary	3
5 Mathematical conventions	4
5.1 Row-vector cell convention	4
5.2 Mixed periodicity	4
6 Public data structures	4
6.1 Display modes	4
6.2 Display roles	4
6.3 Periodic node key	5
6.4 Periodic edge key	5
7 Mode-specific options	5
7.1 Canonical-cell options	5
7.2 Local-unwrapped options	6
7.3 Expanded-cell options	6
7.4 Public option alias	6
8 Prepared periodic graph	7
8.1 Field meaning	7
8.2 Materialized graph constraints	7
9 Public function	7
10 Processing order	8
11 Canonical-cell algorithm	8
12 Local-unwrapped algorithm	8
12.1 Deterministic image assignment	8
12.2 Residual winding	9
12.3 Disconnected source selections	9
13 Expanded-cell algorithm	9
14 Attribute propagation	10
15 Complexity policy	10

16 Metadata and provenance	10
17 Error and warning policy	11
17.1 Errors	11
17.2 Warnings	11
18 Edge cases	11
18.1 Directed graphs	11
18.2 Multigraphs	11
18.3 Self-loops	11
18.4 Nonperiodic graphs	11
18.5 Half-cell and wrapping ambiguity	12
18.6 Coincident images	12
19 Private implementation helpers	12
20 Testing requirements	12
20.1 Key and validation tests	12
20.2 Canonical-cell tests	12
20.3 Local-unwrapped tests	12
20.4 Expanded-cell tests	13
20.5 Complexity and provenance tests	13
20.6 Integration tests	13
21 Performance expectations	13
22 Integration with other modules	14
22.1 Graph-view core	14
22.2 Styling	14
22.3 Two-dimensional renderer	14
22.4 Interactive three-dimensional renderer	14
22.5 Scientific adapters	14
22.6 Future framework and ring adapters	14
23 Implemented source mapping	14
24 Deferred features	14
25 Acceptance checklist	14

1 Purpose and status

This document is the normative specification for implemented Stage G4 in `mdstats.plotting.periodic_graph`. It standardizes renderer-independent materialization of periodic decorated graphs for both static 2-D and interactive 3-D rendering.

The API is implemented and validated in `mdstats` 0.11.0.

2 Motive

A periodic scientific graph is usually stored as a quotient graph. Each canonical node occurs once, while an edge carries an integer translation identifying the periodic image of its target. This representation is compact and authoritative, but it is not directly suitable for human visualization.

A renderer needs explicit display nodes at actual Cartesian positions. Depending on the task, the user may need:

- one canonical cell with ghost endpoints for boundary-crossing edges;
- one continuously unwrapped local neighborhood;

- several replicated cells for global spatial inspection.

These transformations must be shared by all renderers. Otherwise the 2-D and 3-D backends could display different periodic graphs.

The central invariant is

$$\text{scientific periodic graph} \neq \text{materialized display graph}$$

Display replicas and ghosts may multiply graphical objects, but they never become new scientific nodes or edges.

3 Normative ownership

This specification owns:

- periodic display-node and display-edge identities;
- canonical-cell materialization;
- deterministic local unwrapping;
- expanded-cell replication;
- residual winding and boundary ghost handling;
- source-to-display mappings;
- periodic materialization provenance;
- periodic node and edge omission accounting;
- pre-render expansion complexity checks.

It does not own:

- scientific graph construction;
- atomic connectivity or framework projection;
- ring enumeration or ring geometry;
- graph styling;
- 2-D projection;
- Plotly or Matplotlib artists;
- camera, legend, or file-export policy.

4 AI context summary

When modifying this module, preserve the following rules:

1. `DecoratedGraphView` remains the source visualization graph.
2. Periodic preparation runs after source focus and filtering.
3. Source node and edge keys remain authoritative.
4. Every display node is identified by a source node key and an integer image shift.
5. Every display edge is identified by a source edge key and the image shifts of its displayed endpoints.
6. Canonical-cell, local-unwrapped, and expanded views use one common output type.
7. A materialized display edge connects explicit endpoint nodes and therefore carries no unresolved quotient-graph translation.
8. Residual winding is represented by a ghost endpoint rather than by distorting the assigned local embedding.
9. Replication never changes source attributes or scientific metadata.
10. Complexity limits are checked before large arrays are allocated.
11. No display object may be silently omitted.
12. Renderers consume the same prepared periodic graph.

5 Mathematical conventions

5.1 Row-vector cell convention

`mdstats` uses a row-vector cell matrix

$$H = \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix}.$$

For canonical Cartesian node position $\mathbf{x}_i$ and image shift $\mathbf{q} \in \mathbb{Z}^3$, the displayed image position is

$$\mathbf{x}_{i,\mathbf{q}} = \mathbf{x}_i + \mathbf{q}H.$$

A source quotient edge from node i to node j with image shift $\mathbf{m}_{ij}$ represents

$$(i, \mathbf{q}) \longrightarrow (j, \mathbf{q} + \mathbf{m}_{ij})$$

for every lattice image $\mathbf{q}$.

5.2 Mixed periodicity

For every nonperiodic axis α , all source and display image shifts must satisfy

$$q_\alpha = 0, \quad m_{ij,\alpha} = 0.$$

Nonzero translation on a nonperiodic axis is an error.

6 Public data structures

6.1 Display modes

```
from enum import Enum

class PeriodicDisplayMode(Enum):
    CANONICAL_CELL = "canonical_cell"
    LOCAL_UNWRAPPED = "local_unwrapped"
    EXPANDED = "expanded"
```

The mode is recorded in every prepared result.

6.2 Display roles

```
class PeriodicNodeRole(Enum):
    CANONICAL = "canonical"
    REPLICAS = "replicas"
    GHOST = "ghost"

class PeriodicEdgeRole(Enum):
    PRIMARY = "primary"
    REPLICAS = "replicas"
    BOUNDARY_GHOST = "boundary_ghost"
    CYCLE_GHOST = "cycle_ghost"
```

Roles are presentation metadata. They are not scientific classifications.

6.3 Periodic node key

```
@dataclass(frozen=True, slots=True)
class PeriodicNodeKey:
    source_node_key: GraphKey
    image_shift: tuple[int, int, int]
```

The pair

$$(k_i, \mathbf{q}_i)$$

uniquely identifies one displayed image of source node key k_i .

Constraints:

- `source_node_key` must be hashable and present in the source graph;
- `image_shift` must contain exactly three integers;
- shifts on nonperiodic axes must be zero;
- two display nodes with the same key must be deduplicated.

The role is deliberately not part of the key. A node image can be reached first as a replica and later as a boundary endpoint without acquiring a second display identity.

6.4 Periodic edge key

```
@dataclass(frozen=True, slots=True)
class PeriodicEdgeKey:
    source_edge_key: GraphKey
    source_image_shift: tuple[int, int, int]
    target_image_shift: tuple[int, int, int]
```

The edge key preserves source-edge identity and the exact images of both displayed endpoints.

Constraints:

- `source_edge_key` must be present in the source graph;
- both image shifts must contain three integers;
- shifts on nonperiodic axes must be zero;
- keys must be unique after materialization.

For an undirected source graph, endpoint orientation follows the stored source edge. The renderer may draw the segment without an arrow.

7 Mode-specific options

A tagged union is preferred over one dataclass containing fields that are invalid for most modes.

7.1 Canonical-cell options

```
@dataclass(frozen=True, slots=True)
class CanonicalCellDisplay:
    mode: PeriodicDisplayMode = field(
        default=PeriodicDisplayMode.CANONICAL_CELL,
        init=False,
    )
```

The canonical cell contains every selected source node at image $\mathbf{q} = \mathbf{0}$. Boundary-crossing edges materialize deduplicated ghost endpoint images.

For a nonperiodic graph with no quotient translations, a cell is optional and this mode becomes an identity-like explicit materialization. A finite cell is required as soon as any edge carries a nonzero image shift.

7.2 Local-unwrapped options

```
@dataclass(frozen=True, slots=True)
class LocalUnwrappedDisplay:
    center_node_key: GraphKey
    hop_radius: int | None = None
    direction: Literal["both", "out", "in"] = "both"
    mode: PeriodicDisplayMode = field(
        default=PeriodicDisplayMode.LOCAL_UNWRAPPED,
        init=False,
    )
```

Constraints:

- `center_node_key` must survive source focus and filtering;
- `hop_radius`, when supplied, must be a nonnegative integer;
- `direction` follows the graph-distance convention of `GraphFocus`;
- only the connected component containing the center is materialized;
- omitted disconnected components are recorded;
- `node_positions_3d`, `cell`, `pbcs`, and `edge_image_shifts` are required.

When `hop_radius` is `None`, the complete center component is used.

7.3 Expanded-cell options

```
@dataclass(frozen=True, slots=True)
class ExpandedCellDisplay:
    image_ranges: tuple[
        tuple[int, int],
        tuple[int, int],
        tuple[int, int],
    ] = ((0, 0), (0, 0), (0, 0))
    mode: PeriodicDisplayMode = field(
        default=PeriodicDisplayMode.EXPANDED,
        init=False,
    )
```

Each (`lower`, `upper`) interval is inclusive. The number of primary cell images is

$$N_{\text{cell}} = \prod_{\alpha=1}^3 (u_{\alpha} - l_{\alpha} + 1).$$

Constraints:

- every bound must be an integer;
- `lower` ≤ `upper` on every axis;
- nonperiodic axes must use (0, 0);
- the product of interval lengths must be finite and complexity-safe;
- `node_positions_3d`, `cell`, `pbcs`, and `edge_image_shifts` are required.

Boundary endpoints outside the requested primary image range are represented by shared ghost nodes. Source edges are never silently truncated.

7.4 Public option alias

```
PeriodicDisplayOptions = (
    CanonicalCellDisplay
    | LocalUnwrappedDisplay
    | ExpandedCellDisplay
)
```

8 Prepared periodic graph

```
@dataclass(frozen=True, slots=True)
class PeriodicGraphView:
    source_view: DecoratedGraphView
    graph: DecoratedGraphView
    mode: PeriodicDisplayMode

    source_node_positions: NDArray[np.int64]
    source_edge_positions: NDArray[np.int64]
    node_image_shifts: NDArray[np.int64]

    node_roles: tuple[PeriodicNodeRole, ...]
    edge_roles: tuple[PeriodicEdgeRole, ...]

    primary_cell_image_shifts: NDArray[np.int64]

    selection_metadata: Mapping[str, Any]
    periodic_metadata: Mapping[str, Any]
    warnings: tuple[str, ...]
```

8.1 Field meaning

source_view Original immutable `DecoratedGraphView` supplied by the caller.

graph Materialized display graph. Its node keys are `PeriodicNodeKey`; its edge keys are `PeriodicEdgeKey`; its positions are explicit Cartesian display positions.

source_node_positions One-dimensional array of length `graph.n_nodes`. Entry p identifies the source node position in `source_view` for display node p .

source_edge_positions One-dimensional array of length `graph.n_edges`. Entry e identifies the source edge position in `source_view` for display edge e .

node_image_shifts Integer array of shape `(graph.n_nodes, 3)`.

node_roles, edge_roles Role sequences with lengths equal to the display node and edge counts.

primary_cell_image_shifts Integer array of shape `(n_primary_cells, 3)`. Ghost-only images are not included.

selection_metadata Source focus, filtering, and omission records.

periodic_metadata Mode, cell ranges, root selection, ghost counts, replica counts, winding residuals, and source/display count summaries.

8.2 Materialized graph constraints

The `graph` field must satisfy:

- `node_positions_3d` is present and finite;
- `edge_image_shifts` is `None` or an all-zero array;
- every edge endpoint refers to an explicit display node;
- source node and edge attributes are replicated exactly;
- additional display columns include at least `display_role` and `image_shift` for nodes, and `display_role`, `source_image_shift`, and `target_image_shift` for edges;
- display-only metadata never replaces source scientific metadata;
- all arrays are copied, C-contiguous, and read-only.

9 Public function

```
def prepare_periodic_graph_view(
    view: DecoratedGraphView,
    *,
    periodic: PeriodicDisplayOptions | None = None,
```

```

    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
) -> PeriodicGraphView:
    ...

```

periodic=None is equivalent to CanonicalCellDisplay().

10 Processing order

The function must use the following order:

1. validate source graph and option type
2. resolve GraphFocus on the source graph
3. apply GraphFilter to the focused source graph
4. record selected and omitted source objects
5. estimate periodic display counts
6. enforce node and edge complexity limits before allocation
7. materialize the requested periodic mode
8. replicate source metadata
9. validate source-to-display mappings
10. freeze result arrays and provenance

Changing this order can change the meaning of local focus and expansion. For example, replicating before source filtering would multiply objects that the user explicitly excluded.

11 Canonical-cell algorithm

Let the selected source graph have canonical nodes i and edge translations $\mathbf{m}_{ij}$.

1. Materialize every selected source node as $(i, 0)$ with role CANONICAL.
2. For every edge with $\mathbf{m}_{ij} = \mathbf{0}$, connect the two canonical nodes.
3. For every edge with $\mathbf{m}_{ij} \neq \mathbf{0}$, materialize or reuse the target display node $(j, \mathbf{m}_{ij})$ with role GHOST.
4. Connect $(i, 0)$ to $(j, \mathbf{m}_{ij})$ and mark the edge BOUNDARY_GHOST.
5. Deduplicate ghost nodes by PeriodicNodeKey.

The resulting Cartesian edge vector is

$$\mathbf{d}_{ij} = \mathbf{x}_j + \mathbf{m}_{ij}H - \mathbf{x}_i.$$

The mode preserves one canonical source-cell population while making every displayed edge geometrically explicit.

12 Local-unwrapped algorithm

12.1 Deterministic image assignment

Let the selected center be r . Assign

$$\mathbf{q}_r = \mathbf{0}.$$

Traverse the selected center component with deterministic breadth-first search. The neighbor order is source node position followed by source edge position.

For an oriented edge $i \rightarrow j$ with source translation $\mathbf{m}_{ij}$, if j is unassigned, set

$$\mathbf{q}_j = \mathbf{q}_i + \mathbf{m}_{ij}.$$

For reverse traversal, use $-\mathbf{m}_{ij}$.

12.2 Residual winding

If both endpoints are already assigned, define

$$\mathbf{w}_{ij} = \mathbf{q}_i + \mathbf{m}_{ij} - \mathbf{q}_j.$$

When $\mathbf{w}_{ij} = \mathbf{0}$, the edge connects the assigned display nodes.

When $\mathbf{w}_{ij} \neq \mathbf{0}$, the cycle carries residual periodic winding. The assigned local embedding must not be modified to force closure. Instead, materialize or reuse a ghost image of node j at

$$\mathbf{q}_j^{\text{ghost}} = \mathbf{q}_i + \mathbf{m}_{ij},$$

and connect the edge to that ghost with role `CYCLE_GHOST`.

This is the correct diagnostic behavior. A winding cycle cannot be represented as a single closed Euclidean cycle inside one finite unwrapped embedding without cutting at least one edge.

12.3 Disconnected source selections

Only the center component is materialized. Other selected components are omitted with an explicit count and source-key list or digest in `selection_metadata`.

The function must not place unrelated components at the same origin or invent arbitrary component translations.

13 Expanded-cell algorithm

Let

$$Q = \{(q_1, q_2, q_3) : l_\alpha \leq q_\alpha \leq u_\alpha\}$$

be the requested primary image set.

1. For every selected source node i and every $\mathbf{q} \in Q$, materialize $(i, \mathbf{q})$.
2. Nodes at $\mathbf{q} = \mathbf{0}$ have role `CANONICAL`; other primary nodes have role `REPLICA`.
3. For every selected source edge $i \rightarrow j$ with shift $\mathbf{m}_{ij}$ and every $\mathbf{q} \in Q$, construct

$$(i, \mathbf{q}) \longrightarrow (j, \mathbf{q} + \mathbf{m}_{ij}).$$

4. If the target image lies outside Q , materialize or reuse a ghost endpoint.
5. Use the source-cell convention: exactly one display edge is generated per source edge per primary image.
6. Deduplicate display nodes and validate unique `PeriodicEdgeKey` values.

Before ghosts, the primary counts are

$$N_{\text{node}}^{\text{primary}} = N_V N_{\text{cell}}, \quad N_{\text{edge}}^{\text{primary}} = N_E N_{\text{cell}}.$$

Ghost-node count depends on boundary translations and must be estimated conservatively before allocation.

14 Attribute propagation

Every source node attribute column is indexed by `source_node_positions` and copied to the display graph. Every source edge attribute column is indexed by `source_edge_positions`.

The periodic module adds display metadata but must not rewrite source values.

Recommended display columns are:

```
node attributes
    display_role      str
    image_shift       tuple[int, int, int]

edge attributes
    display_role      str
    source_image_shift tuple[int, int, int]
    target_image_shift tuple[int, int, int]
```

Style rules may use these columns to fade ghosts or replicas.

15 Complexity policy

The existing `GraphComplexityPolicy.max_nodes` and `max_edges` apply to display counts, not only to source counts.

The periodic module must estimate at least:

- selected source nodes and edges;
- primary display nodes and edges;
- conservative maximum ghost nodes;
- final display nodes and edges after materialization.

When a limit is exceeded:

- `overflow="error"` raises `GraphComplexityError`;
- `overflow="require_focus"` raises an error explaining which focus or image range should be reduced;
- `overflow="warn_and_render"` records a warning and proceeds.

The module must not silently reduce image ranges, hop radius, or graph content.

16 Metadata and provenance

`periodic_metadata` must include:

```
mode
source node and edge counts
selected source node and edge counts
display node and edge counts
canonical, replica, and ghost node counts
primary, replica, boundary-ghost, and cycle-ghost edge counts
primary cell image shifts
center node key and hop radius, when applicable
expanded image ranges, when applicable
residual winding vectors, when present
cell and PBC summary
materialization schema version
warnings
```

A stable schema identifier is recommended:

```
PERIODIC_GRAPH_DISPLAY_SCHEMA = "mdstats.periodic-graph-display.v1"
```

Display provenance is not part of scientific graph identity.

17 Error and warning policy

17.1 Errors

Raise a graph-visualization exception for:

- missing physical positions, cell, PBC flags, or edge shifts when required;
- nonfinite positions or cells;
- singular cells;
- image shifts of incorrect shape or dtype;
- nonzero shifts along nonperiodic axes;
- unknown local center node;
- invalid hop radius or direction;
- invalid or empty expanded image ranges;
- duplicate source keys that violate `DecoratedGraphView`;
- source edges whose image translations cannot be reconciled with endpoint arrays;
- unsupported self-loops when they require an unimplemented display path;
- complexity overflow under a rejecting policy;
- inconsistent source-to-display mappings.

17.2 Warnings

Record warnings for:

- minimum-image ties already reported by an adapter;
- residual winding requiring cycle ghosts;
- large ghost-to-primary node ratios;
- a local selection that omits disconnected selected components;
- expanded ranges that produce a very anisotropic display;
- a graph with no edges;
- display-node overlap at identical Cartesian coordinates.

Warnings must not change graph content.

18 Edge cases

18.1 Directed graphs

Image propagation follows edge orientation. `direction` controls local graph distance, but a displayed directed edge retains its original source and target. The periodic module does not add arrowheads.

18.2 Multigraphs

Parallel source edges remain distinct because `PeriodicEdgeKey` contains the stable source edge key. Display-node deduplication must not collapse parallel edges.

18.3 Self-loops

A zero-translation self-loop has no straight-line extent and is deferred unless a renderer explicitly supports loop glyphs. A nonzero periodic self-edge can be materialized between two images of the same source node, but the first G4 implementation may reject it with `GraphUnsupportedFeatureError`.

18.4 Nonperiodic graphs

A graph with all `pbc=False` and zero edge translations is valid. Canonical-cell materialization becomes an identity-like explicit graph with no replicas or ghosts. Expanded ranges must remain (0, 0) on all axes.

18.5 Half-cell and wrapping ambiguity

The periodic module does not infer minimum-image shifts. It trusts the source view's frame-consistent `edge_image_shifts`. Ambiguous reconstruction belongs to the scientific adapter and must already be recorded there.

18.6 Coincident images

Different periodic node keys can occupy the same Cartesian position in degenerate or pathological cells. They remain distinct display objects. A warning should be emitted.

19 Private implementation helpers

The following behavior should remain private and replaceable:

```
source selection dispatch
integer image-range enumeration
ghost-node interning
source-to-display reindexing
deterministic adjacency construction
local BFS image assignment
residual winding detection
expanded-edge generation
attribute replication
periodic complexity estimation
schema serialization
```

Renderers must not depend on private helper names.

20 Testing requirements

20.1 Key and validation tests

- periodic node and edge keys are hashable and deterministic;
- malformed image shifts fail;
- nonperiodic-axis translations fail;
- result arrays are immutable;
- source mappings have exact lengths and valid positions.

20.2 Canonical-cell tests

- zero-shift edges use canonical endpoints;
- crossing edges create correct ghost endpoints;
- multiple edges reuse the same ghost node image;
- source attributes are preserved;
- no source object is modified.

20.3 Local-unwrapped tests

- a periodic chain is embedded continuously;
- deterministic BFS is independent of input dictionary order;
- reverse edge traversal uses the opposite shift;
- winding cycles create residual ghost endpoints;
- a zero-winding cycle closes without a ghost;
- hop radius and direction match **GraphFocus** semantics;
- disconnected components are explicitly omitted.

20.4 Expanded-cell tests

- primary node and edge counts scale with cell count;
- boundary ghosts are correct;
- mixed periodic axes reject invalid ranges;
- no duplicate display edge keys occur;
- canonical, replica, and ghost roles are correct;
- source-cell replication produces each quotient edge once per primary cell.

20.5 Complexity and provenance tests

- overflow modes behave exactly as documented;
- estimates are conservative;
- metadata counts match arrays;
- source and display identities round-trip through mappings.

20.6 Integration tests

The relaxed Na-LTA fixture must support:

- one-cell canonical framework display;
- a local unwrapped Si-centered neighborhood;
- an expanded 2 x 2 x 1 framework display;
- framework plus Na-O diagnostic contacts;
- identical prepared geometry for 2-D and 3-D consumers.

The authoritative framework fixture remains:

```
144 framework atoms
192 T-O edges
48 T atoms with degree 4
96 framework O atoms with degree 2
```

21 Performance expectations

Let N_V and N_E be selected source counts and N_C the number of primary cell images.

Canonical-cell preparation is

$$O(N_V + N_E).$$

Local unwrapping is

$$O(N_V + N_E)$$

for the selected center component.

Expanded preparation is

$$O(N_C(N_V + N_E))$$

plus ghost-node interning. Hash-based key interning should be expected average $O(1)$ per display object.

The implementation should use preallocated or append-once Python lists followed by single NumPy array construction. Repeated array concatenation is prohibited.

22 Integration with other modules

22.1 Graph-view core

G4 consumes source focus, filters, and complexity policy. It does not redefine those classes.

22.2 Styling

Styles resolve against replicated display attributes. Ghost and replica appearance can be controlled through `display_role` rules.

22.3 Two-dimensional renderer

In `mdstats` 0.11.0, generalized periodic materialization and deterministic local image assignment are owned here. The 2-D renderer exposes an explicit `periodic=` path that consumes `PeriodicGraphView`; its legacy 0.10.0 controls remain backward-compatible and reuse the G4 image-assignment helper.

22.4 Interactive three-dimensional renderer

G5 consumes `PeriodicGraphView` and must not independently replicate or unwrap the source graph.

22.5 Scientific adapters

Adapters remain responsible for frame-consistent source positions and quotient edge translations. G4 trusts those inputs and performs no scientific connectivity test.

22.6 Future framework and ring adapters

Projected framework vertices, ring centers, site centers, and cage centers can all be replicated through the same source/display mapping. The periodic module must remain agnostic to their scientific meaning.

23 Implemented source mapping

The public implementation resides in `mdstats.plotting.periodic_graph`. The canonical-cell, local-unwrapped, and expanded modes all construct one immutable `PeriodicGraphView`. The 2-D and 3-D renderers consume the same image-assignment and source/display identity contracts.

24 Deferred features

The first G4 implementation defers:

- arbitrary spatial clipping surfaces;
- edge clipping against cell faces;
- periodic Voronoi regions;
- automatic image-range selection from a camera;
- graph coarsening;
- trajectory-dependent replica persistence;
- nonlinear cell deformation between frames;
- self-loop glyph construction;
- distributed or out-of-core replication.

25 Acceptance checklist

The G4 implementation is accepted when:

- every public field and function has one normative owner;
- mode-specific options cannot express invalid cross-mode combinations;

- canonical-cell, local, and expanded algorithms are deterministic;
- winding cycles preserve topology through explicit ghost endpoints;
- source-to-display mappings are complete and immutable;
- 2-D and 3-D renderers can consume one prepared result;
- complexity is checked before expansion allocation;
- Na-LTA acceptance geometries are specified;
- no scientific graph identity is modified or recreated.